Money Tracker

Project documentation and team plan

Architecture, decisions, data model, API, security, scope, owner tasks, FAQs and test cases

Built by Team G

Microsoft Innovations Club, VIT Chennai

A personal finance PWA for hostel students, with SMS capture on Android.

Version 0.1 | October 2026

Contents

1. Overview . . . 3
2. Scope and phases . . . 4
3. Architecture . . . 5
4. Technology decisions and why . . . 7
5. Repository structure . . . 9
6. Data model . . . 10
7. API reference . . . 11
8. Security model . . . 12
9. SMS parsing and duplicate detection . . . 13
10. Development setup and workflow . . . 14
11. Current status . . . 15
12. Pending work by owner . . . 16
13. FAQs . . . 18
14. Test cases . . . 20
15. Risks and open decisions . . . 24

1. Overview

Money Tracker is a personal finance app for hostel students. It records income and expenses, sorts them into categories that match hostel life, tracks budgets and savings goals, and shows everything on an interactive dashboard. It installs as a PWA on laptops and phones. On Android an APK adds automatic SMS reading, so past bank transactions are filled in without typing.

Why this exists

Hostel students live on a fixed allowance. Money disappears in small daily spends (mess, canteen, recharge, printouts) that no one writes down. Existing apps are built for salaried adults and ask for too much typing. This app targets that gap with hostel categories, a budget cycle that starts on allowance day, and automatic capture.

Key promises

- Recording money takes a few seconds, and often no time at all because it is captured automatically.
- One account works on web, Android and iPhone.
- Raw SMS text never leaves the phone. Only the extracted fields (amount, merchant, last 4 digits, reference) are sent.
- Works offline and syncs when the connection returns.
- Anything the app is unsure about goes to a review list. Nothing is saved silently or dropped.

Platform support

Capability	Web	iPhone (PWA)	Android (APK)
Manual entry, budgets, goals, dashboard	Yes	Yes	Yes
SMS capture	No	No (iOS blocks it)	Yes
Gmail bank email capture	Yes	Yes	Yes
Bill scan (OCR)	Upload	Camera or upload	Camera or upload
Offline entry	Yes	Yes	Yes
Push notifications	Yes	After Add to Home Screen	Yes (FCM)

2. Scope and phases

Work is delivered in three phases. Each phase ends with working software, so the team always has something to demo.

Phase	In scope	Out of scope
1. MVP	Auth, accounts, hostel categories, manual transactions, edit, delete, search, budgets with alerts, savings goals, dashboard and insights, PWA install, push notifications, CSV export, account deletion	Splits, group budgets, OCR, Gmail
2. Capture	Offline entry and sync, recurring entries, Android APK, SMS history scan and live listener, review list, duplicate detection, merchant to category learning	SMS on iPhone (not possible)
3. Extras	Gmail bank email capture, bill scan with Textract, split expenses and IOUs, shared group budgets	Other currencies, Play Store release, non-Gmail email

Hard constraints

- Money is stored as integer paise. INR only.
- Times are stored in UTC and shown in local time.
- Unreadable captures go to a review list.
- The APK is shared inside the college, since Google Play restricts the SMS permission.
- Gmail access stays in Google test mode for now: limited test users, and refresh tokens expire after 7 days.

Success looks like

- A student installs the PWA, signs up, and logs an expense in under 10 seconds.
- An Android user grants SMS access and sees their last months of bank transactions in the app.
- Budget alerts arrive as push notifications at 50%, 80% and 100%.
- CI is green on main and any teammate can deploy through the pipeline.

3. Architecture

Three clients talk to one API over HTTPS. The API and a small worker run as Docker containers on one EC2 instance behind Nginx. Data lives in RDS PostgreSQL.

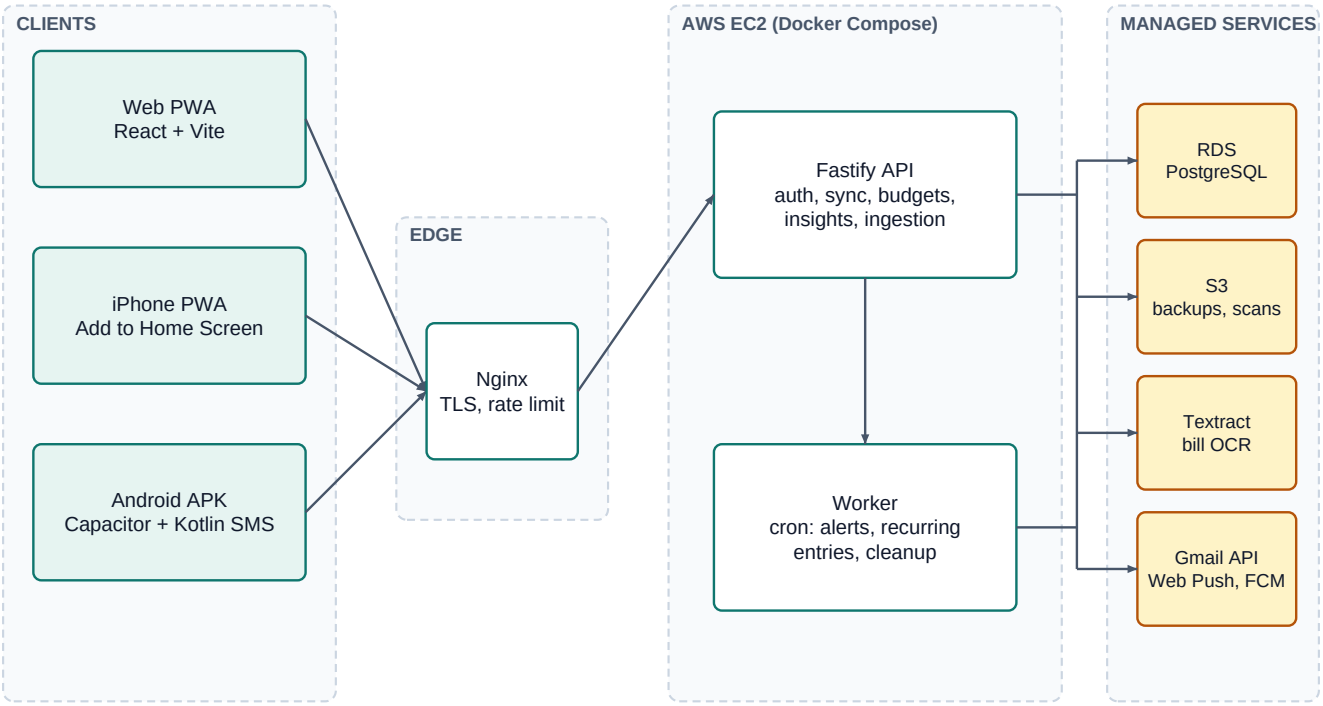


Figure 1. System architecture

How a request flows

- The client sends HTTPS with an **Authorization: Bearer** token. Nginx terminates TLS and applies a request rate limit.
- Fastify validates the body with Zod, checks the token, and runs the route handler.
- Prisma reads or writes PostgreSQL. Every query is scoped by the user id from the token.
- The worker runs scheduled jobs (budget alerts, recurring entries) against the same database. Jobs are safe to run twice.

Components

Component	Responsibility	Owner
Web PWA	UI, offline storage, install, web push	Frontend 1 and 2
Android APK	Same web app in Capacitor, plus the Kotlin SMS plugin	Android
Fastify API	Auth, transactions, sync, budgets, insights, ingestion	Backend 1 and 2
Worker	Scheduled jobs: alerts, recurring entries, cleanup	Backend 2 and DevOps
Nginx	TLS, redirects, rate limiting	DevOps
RDS PostgreSQL	All persistent data, automated backups	DevOps
S3	Database backups and temporary scan images	DevOps
Textract, Gmail API, FCM	Bill OCR, bank email capture, Android push	Backend 2

SMS capture flow

Only the Android app can read SMS. The Kotlin plugin filters by bank sender id, so OTPs and personal chats never reach JavaScript. The shared TypeScript parser extracts fields, and only those fields go to the server.

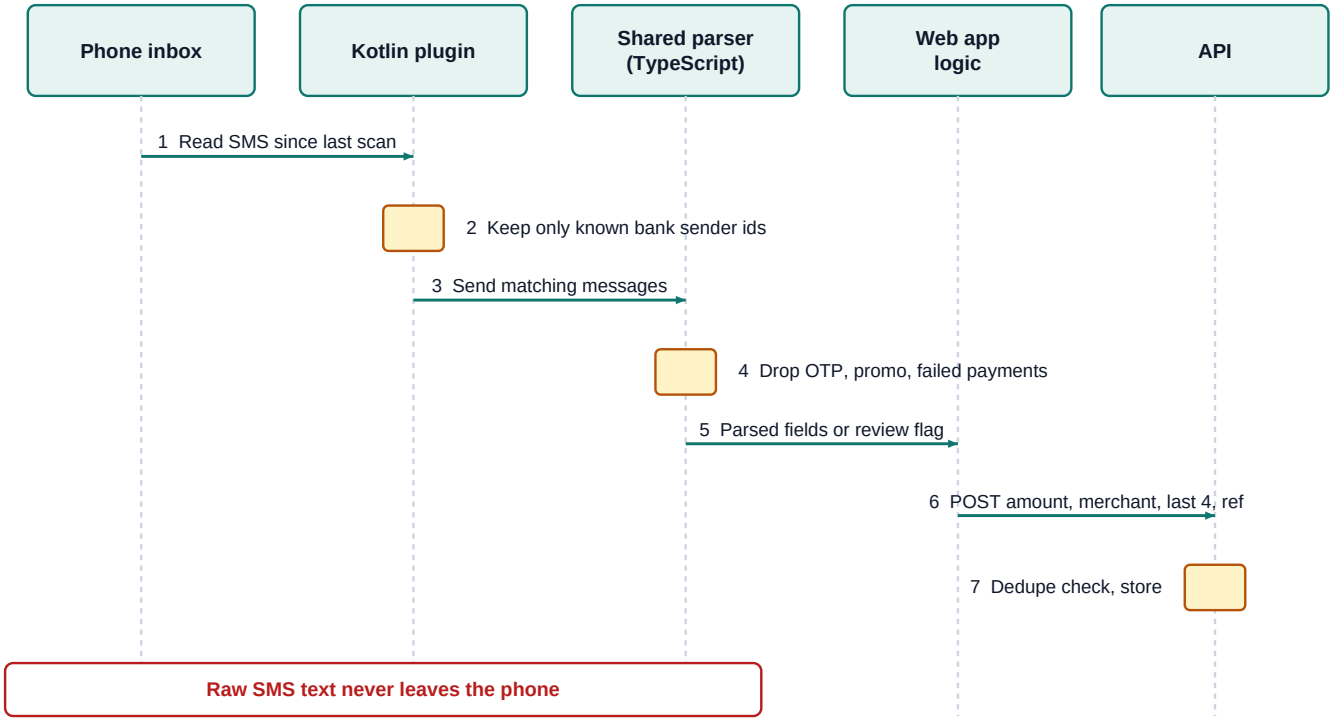


Figure 2. SMS capture, step by step

Offline entry and sync

The phone creates the transaction id, so a retried request can never create a second record. Offline entries wait in IndexedDB until the network returns.

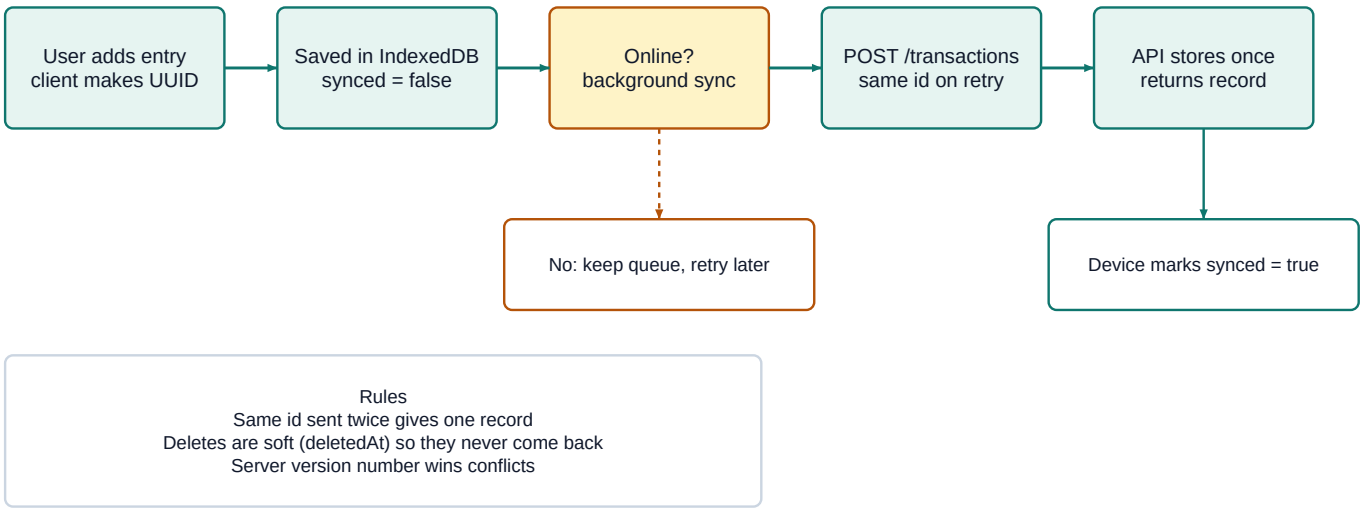


Figure 3. Offline entry and sync

4. Technology decisions and why

Every choice below was made for a reason. If a reason stops being true, the decision should be revisited and logged in the repository.

Decision	Chosen	Alternatives	Why
Repo layout	pnpm monorepo: apps/api, apps/web, apps/android, packages/shared	Separate repos	One pull request can change the API, the shared types and both clients together. Types cannot drift.
Web framework	React 19 + Vite SPA, static build	Next.js	Capacitor packages a static build. A static build has no server, so Next.js API routes would not work in the APK. One build serves web and Android.
API	Separate Fastify 5 service	Next.js route handlers, Express, NestJS	One API serves browser and APK the same way. Fastify is fast, small, and has first-class TypeScript support. NestJS is heavier than this team needs.
Validation	Zod, shared package	Manual checks, Joi	The same schema validates on the phone and the server. Bad input is rejected before it reaches a handler.
Database	PostgreSQL on RDS	MongoDB, SQLite	Money data is relational and needs constraints and transactions. RDS gives automated backups.
ORM	Prisma 6	Drizzle, raw SQL	Typed queries, easy migrations, readable schema for a team of six. Pinned to v6 for stable setup.
Money type	Integer paise	Float rupees, decimal	Floats drift (0.1 + 0.2). Integers are exact and fast. INR only keeps it simple.
Password hashing	Argon2id (@node-rs/argon2)	bcrypt	Memory-hard, stronger against GPU attacks. Prebuilt binaries, so no compiler setup on Windows.
Sessions	15 min JWT + rotating refresh token	Cookies, long JWT	Header tokens behave the same in browser and APK. Short access tokens limit damage if stolen. Reused refresh token revokes all sessions.
Offline storage	IndexedDB via Dexie	localStorage	localStorage is small and synchronous. IndexedDB stores thousands of entries and queries by date.
Android app	Capacitor + Kotlin plugin	React Native, Flutter, TWA	Reuses the web code. A TWA cannot read SMS. Native work is limited to one small plugin.
SMS parsing	On the device, TypeScript	Server-side parsing	Privacy: raw text never leaves the phone. The same engine also runs in tests.
SMS rules	Data records served by API	Hard-coded patterns	A bank changes its format, we update a record. No new APK. Patterns are length-capped against regex abuse.
Duplicates	Reference match, else 5 minute window flagged for review	Hash to the minute	A minute boundary splits two copies of one payment. A window plus a review flag never merges two real payments silently.
Sync conflicts	Server version number	Phone timestamps	Phone clocks are wrong often. The server decides order.
Distribution	Sideloaded APK	Google Play	Play restricts the SMS permission. iPhone users install the PWA, so no app store account is needed.
Hosting	EC2 + Docker Compose + RDS (Mumbai)	Lambda, ECS	Required by the team. Simple to run and debug. Mumbai region is closest to users and keeps data in India.
Edge	Nginx with Let's Encrypt	ALB	Cheap TLS and rate limiting on one box. Move to ALB if traffic grows.
Bill OCR	AWS Textract	Tesseract, Google Vision	Managed, accurate on receipts, stays inside AWS.

Decision	Chosen	Alternatives	Why
Email capture	Gmail API, read-only	IMAP	OAuth, no password handling, and iPhone users get automatic tracking.
Push	Web Push + FCM	One provider	Browsers and iOS PWAs use Web Push. Android WebView needs FCM.
Testing	Vitest, later Playwright	Jest	Vitest shares Vite config and runs fast. Playwright for end to end flows.
CI/CD	GitHub Actions	Jenkins	Lives with the code, free for the team, builds the APK too.

5. Repository structure

Each folder has one owner. The shared package is the contract between all apps, so changes to it need review from the Android owner and one web owner.

```
money-tracker/
|-- apps/
|   |-- api/                                Fastify API
|   |   |-- prisma/schema.prisma            Database schema (source of truth)
|   |   |-- src/server.ts                   Boots the app, graceful shutdown
|   |   |-- src/app.ts                      Plugins: helmet, cors, rate limit, jwt, errors
|   |   |-- src/env.ts                      Validates environment variables
|   |   |-- src/tokens.ts                   Refresh token generation and hashing
|   |   |-- src/routes/auth.ts              register, login, refresh, logout
|   |   |-- src/routes/transactions.ts      create, list, delete
|   |   |-- src/app.test.ts                 API tests
|   |   |-- Dockerfile                      Production image
|   |   `-- .env.example                    Environment template
|   |-- web/                                React PWA
|   |   |-- src/App.tsx, main.tsx           Routes and entry
|   |   |-- src/pages/                      Login, Dashboard
|   |   |-- src/lib/api.ts                  Fetch with token refresh
|   |   |-- src/lib/db.ts                   Dexie (IndexedDB) schema
|   |   |-- src/lib/sms.ts                  Android SMS bridge
|   |   |-- public/                         PWA icons
|   |   `-- vite.config.ts                  Tailwind and PWA manifest
|   `-- android/                            Capacitor project
|       |-- capacitor.config.json           App id and web build folder
|       `-- android/.../SmsReaderPlugin.kt  Kotlin SMS plugin
|-- packages/shared/src/                    Contract between all apps
|   |-- money.ts                            Rupees to paise, formatting
|   |-- schemas.ts                          Zod request schemas
|   |-- dedupe.ts                           Duplicate comparison
|   `-- sms/                                Engine, rules, types
|-- infra/                                  Deployment files
|   |-- docker-compose.yml                  Local Postgres
|   |-- docker-compose.prod.yml             API + Nginx on EC2
|   `-- nginx.conf                          TLS, rate limit, proxy
|-- docs/                                   This document
|-- .github/workflows/ci.yml                Lint, typecheck, test, build, APK
|-- CLAUDE.md                               Rules and facts for AI assistants
`-- README.md                               Overview and quick start
```

Why this layout

- **packages/shared** holds anything the phone and the server must agree on: types, the SMS parser, money helpers, duplicate logic.
- **apps/android** only wraps the web build. Almost all logic stays in TypeScript, so there is one code path to test.
- **infra** keeps deployment files next to the code, so a deploy is reproducible from the repository.

6. Data model

PostgreSQL through Prisma. The schema file is the source of truth. Money columns end in Paise and hold whole numbers.

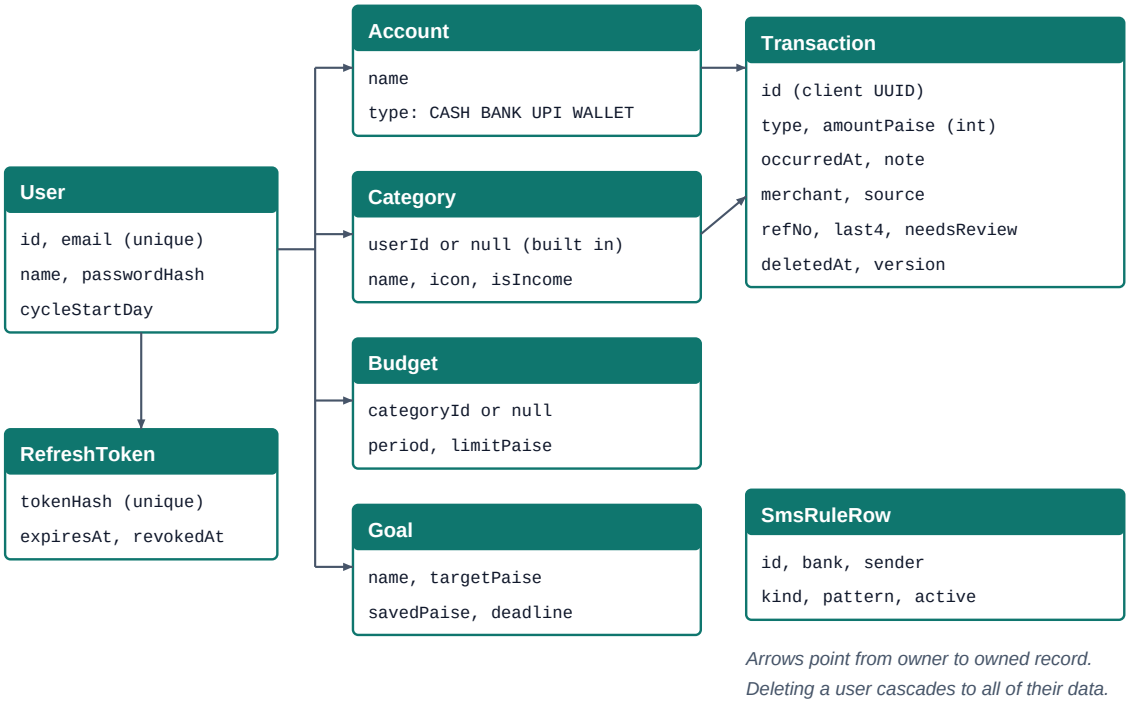


Figure 4. Entities and relations

Why the schema looks like this

Choice	Reason
Transaction.id is a client UUID	A retried sync cannot create a duplicate. The server never invents ids for synced entries.
deletedAt instead of delete	Deleted entries must not come back from another device during sync.
version on Transaction	The server bumps it on every change. Higher version wins conflicts.
Unique (userId, refNo)	The same bank reference cannot be stored twice for one user. Duplicate SMS and email collapse.
needsReview flag	Uncertain captures wait for the user instead of polluting totals.
Category.userId nullable	Built-in hostel categories are shared. Users add private ones.
RefreshToken stores a hash	A database leak does not give attackers working tokens.
Indexes on (userId, occurredAt)	Dashboard and list queries filter by user and date range.
Cascade on user delete	Deleting an account removes all of that user's data, as promised in the privacy rules.

7. API reference

Base path **/api/v1**. Versioned so older APKs keep working after the server changes. All bodies are JSON and validated with Zod. Authenticated routes need **Authorization: Bearer <accessToken>**.

Method and path	Auth	Purpose	Notes	Status
GET /health	No	Liveness check	Used by Docker and CI	Done
POST /auth/register	No	Create account	10 per minute. Creates a Cash account. 409 if email exists	Done
POST /auth/login	No	Get tokens	10 per minute. Same response for wrong email or password	Done
POST /auth/refresh	No	Rotate tokens	Old token revoked. Reuse revokes all sessions	Done
POST /auth/logout	No	Revoke refresh token	204	Done
GET /transactions	Yes	List own transactions	Filters: from, to, limit up to 200	Done
POST /transactions	Yes	Create (idempotent)	Same id returns stored record. 409 on id or reference clash	Done
DELETE /transactions/:id	Yes	Soft delete	404 if not yours	Done
PUT /transactions/:id	Yes	Edit	Bumps version	Pending
GET /sync/changes?cursor	Yes	Change feed	Includes tombstones	Pending
GET/POST /accounts, /categories	Yes	Manage accounts and categories		Pending
GET/POST /budgets, /goals	Yes	Budgets and goals		Pending
GET /insights	Yes	Safe to spend, runway, patterns		Pending
GET /sms-rules	Yes	Bank parsing rules	Versioned, validated by smsRuleSchema	Pending
POST /sms/ingest	Yes	Parsed SMS fields	Never raw text	Pending
GET /review, POST /review/:id	Yes	Review list	Accept or reject uncertain captures	Pending
POST /push/subscribe	Yes	Register Web Push or FCM token		Pending
GET /export.csv, DELETE /me	Yes	Export data, delete account		Pending

Error format

```
{ "error": "invalid_request", "issues": [ ... ] }      400, from Zod
{ "error": "unauthorized" }                           401
{ "error": "email_taken" }                            409
{ "error": "server_error" }                           500, details only in logs
```

8. Security model

Security was designed in from the start, since the app holds financial data and reads SMS.

Threat	Control	Where
Stolen password database	Argon2id hashes. Refresh tokens stored as SHA-256 hashes.	auth.ts, tokens.ts
Account enumeration	Same 401 for unknown email and wrong password. A dummy hash is verified so timing matches.	auth.ts
Brute force	10 requests per minute on auth routes, 120 per minute overall, Nginx limit on top.	app.ts, nginx.conf
Stolen access token	15 minute lifetime.	app.ts
Stolen refresh token	Rotation on every use. A reused token revokes every session for that user.	auth.ts
Malformed or hostile input	Zod on every body and query. 100 KB body limit. Length caps on every text field.	shared/schemas.ts
Reading another user's data	Every query filters by the user id from the token. Foreign ids return 404 or 409 without leaking.	transactions.ts
Cross-site requests	CORS allowlist from environment. Bearer tokens, no cookies, so no CSRF.	app.ts
Browser attacks	Helmet security headers. HSTS from Nginx.	app.ts, nginx.conf
Regex abuse through rules	Rule patterns capped at 500 chars and message length at 600. Rules come only from our API.	shared/sms
Privacy of SMS	Kotlin filters by bank sender, parser runs on device, only fields are uploaded.	SmsReaderPlugin.kt
Secrets in git	.env ignored. Secrets in AWS SSM. CI runs a dependency audit.	.gitignore, ci.yml
Database exposure	RDS private, reachable only from the EC2 security group.	infra (DevOps)
Weak server config	TLS 1.2+, container runs as non-root user, logs redact Authorization header.	Dockerfile, app.ts

> Known gaps to close before public launch: Android secure storage for the refresh token, DPDP Act consent for users under 18, Google verification for Gmail, and a penetration check of the review and sync endpoints.

9. SMS parsing and duplicate detection

Pipeline in packages/shared/src/sms/engine.ts

Step	Check	Result if it fails
1	Sender id matches a known bank or UPI sender	Ignored
2	Message is 600 characters or fewer	Ignored
3	Not an OTP or promo (unless money actually moved)	Ignored
4	Not failed, declined, reversed or insufficient funds	Ignored
5	Contains debited, credited, sent, received or paid	Ignored
6	A bank rule matches and the amount is valid	Parsed
7	No rule matches, or amount unreadable	Review list

Starter rules are included for HDFC and SBI debits and a generic credit. Real SMS formats vary, so the Android owner extends them from an anonymised message corpus gathered from the team.

Duplicate verdicts (packages/shared/src/dedupe.ts)

Condition	Verdict	What happens
Amount or type differs	DISTINCT	Stored as separate entries
Both have a reference number and it matches	DUPLICATE	Second copy dropped, kept one record
Both have a reference number and it differs	DISTINCT	Separate payments
No reference, within 5 minutes, last 4 digits not conflicting	POSSIBLE	Stored and flagged for review
No reference, more than 5 minutes apart	DISTINCT	Separate entries

Edge cases handled

- **Bank changes its format:** messages go to review. A rule update fixes it for everyone.
- **Refund or reversal:** creates an income entry linked to the original expense (pending in backend work).
- **Permission denied:** the app works fully with manual entry, email and scans.
- **Non-INR amount:** sent to review.
- **Android 13+ sideload:** the user must allow restricted settings once. Onboarding explains the steps.
- **Battery savers:** a catch-up scan runs every time the app opens, so missed messages are still found.

10. Development setup and workflow

Requirements

- Node 22 and pnpm 10
- Docker Desktop for local Postgres
- JDK 21 and Android SDK for the APK

First run

```
pnpm install
cp apps/api/.env.example apps/api/.env
cp apps/web/.env.example apps/web/.env
docker compose -f infra/docker-compose.yml up -d
pnpm --filter @money-tracker/api db:migrate
pnpm dev:api     # http://localhost:4000
pnpm dev:web     # http://localhost:5173
```

Daily commands

Command	What it does
pnpm test	Runs Vitest in every package
pnpm typecheck	TypeScript check across the repo
pnpm lint	ESLint over all source
pnpm build	Production builds
pnpm --filter @money-tracker/api db:generate	Regenerate Prisma client after schema edits
pnpm --filter @money-tracker/android sync	Copy web build into the Android project
./gradlew assembleDebug (in apps/android/android)	Build the debug APK

Team rules

- Short-lived feature branches, merged to main through pull requests. No direct pushes to main.
- Every pull request needs one approving review from another developer and a green CI run.
- API contract changes go into packages/shared first, with approval from the Android owner and a web owner.
- Each area has one owner who keeps its README current. Cross-area decisions go in a decisions log in the repository.
- Environments: development and production, deployed only through CI. Secrets live in AWS, never in git.
- Cadence: weekly planning, a mid-week check-in, a weekly demo of working software.

11. Current status

Area	Done	Not done
Repo	pnpm monorepo, lint, typecheck, CI, Docker files, README, CLAUDE.md	Terraform, branch protection
Shared	Money helpers, Zod schemas, SMS engine with 3 starter rules, dedupe, 9 tests	More bank rules from a real SMS corpus
API	Health, register, login, refresh, logout, create, list and delete transactions, security headers, 7 tests	Categories, budgets, goals, sync, insights, push, real database tests, migrations
Web	PWA shell, login page, dashboard chart, IndexedDB schema, API client with token refresh, production build	Entry form, lists, budgets, goals, settings, sync, install prompt, push, shadcn/ui
Android	Capacitor project, Kotlin SMS plugin, READ_SMS permission, TypeScript bridge, debug APK builds	Live SMS listener, FCM, signing, restricted settings onboarding, device testing

Verified so far: lint clean, typecheck clean, 16 automated tests passing, web production build passing, Android debug APK built (4.4 MB). The APK has not yet been run on a phone.

12. Pending work by owner

Each person owns an area end to end. Cross-area work is agreed between the two owners first.

Frontend 1: entry, offline, PWA

- Transaction entry form (amount, category, account, date, note) with a fast numeric keypad flow
- Transaction list with search and filters (date, category, account, amount), edit and delete
- Offline queue in Dexie and sync client: retry, idempotent ids, conflict handling
- PWA install prompt, iPhone Add to Home Screen guidance, web push subscription
- Bill upload screen and pre-filled confirm form (phase 3)
- Add shadcn/ui and agree shared components with Frontend 2

Frontend 2: dashboard and money screens

- Dashboard: category split, daily and weekly trend, budget progress, month comparison
- Safe to spend today, runway and goal pace cards
- Budget screens: overall and per category, weekly or monthly, custom cycle start day
- Savings goals screens with amount per day
- Settings: profile, PIN lock, CSV export, delete account, Gmail connect screen
- Review list screen for uncertain captures

Backend 1: core API and sync

- Prisma migrations and seed data for hostel categories and income types
- Accounts and categories endpoints, custom categories
- Transactions: update, search, filters, transfers between accounts
- Sync endpoints: change feed with cursor, tombstones, server version conflicts
- Duplicate detection service using the shared rules, review queue API
- SMS ingestion endpoint and merchant to category learning
- Serve SMS rules with versioning

Backend 2: budgets, insights, integrations

- Budgets and goals endpoints, budget cycle maths
- Insights: safe to spend, runway, pattern detection, month over month
- Notification service: threshold alerts once per period, weekly summary, Web Push and FCM
- Recurring entries engine (idempotent)
- Gmail OAuth and bank email fetching (phase 3)
- Textract integration and scan cleanup (phase 3)
- Splits, IOUs and group budgets (phase 3)

Android: native app and parsing rules

- Confirm the APK builds on a clean machine, add release signing
- SMS live listener (BroadcastReceiver) and catch-up scan on app open
- Onboarding for SMS permission, including the Android 13+ restricted settings step
- Collect an anonymised SMS corpus from the team and write rules for each major bank
- Parser test suite per bank, review list hand-off
- FCM push, biometric unlock, secure storage for the refresh token
- Test on several phone brands for battery killers, in-app update prompt

Cloud and DevOps

- AWS setup in Mumbai (ap-south-1): VPC, EC2, RDS, S3, security groups, least-privilege IAM
- Domain and HTTPS with Let's Encrypt, Nginx config from infra/
- Secrets in SSM Parameter Store, wired into Docker Compose
- GitHub Actions deploy to dev and prod, branch protection, required reviews
- Worker container for cron jobs, scheduled RDS and S3 backups, restore test
- CloudWatch logs and alarms, Amazon SES for password reset and verification
- Static web hosting (S3 and CloudFront) and APK release pipeline

13. FAQs

Q. Why not read SMS in the PWA?

Browsers cannot read SMS. Only a native app can, which is why the Android APK exists. iPhone gets email capture and bill scanning instead.

Q. Why is the API separate from the web app?

The APK needs a static web build, and a static build has no server. One standalone API serves browser and APK the same way.

Q. Is my SMS uploaded?

No. The phone reads and parses messages locally. Only amount, merchant, last 4 digits and reference are sent. OTPs and promotions are never read because the plugin filters by bank sender id.

Q. What if a bank changes its SMS format?

Unreadable messages go to the review list. We fix it by updating the rule data on the server, with no new APK needed.

Q. How are duplicates handled?

The same payment can arrive by SMS, email and manual entry. A matching reference number means duplicate. Otherwise a close time and amount match is flagged for review.

Q. What happens offline?

Entries are saved on the device with their own id and synced later. Retrying never creates duplicates.

Q. Why do iPhone push notifications need Add to Home Screen?

iOS only allows web push for installed PWAs.

Q. Why can the SMS permission be blocked on the APK?

Android 13 and later restricts sensitive permissions for apps installed outside the Play Store. The user allows it once in app settings. Onboarding explains the steps.

Q. Can Gmail capture go public?

Not without Google's verification and security review for restricted scopes. In test mode only listed users work and tokens expire every 7 days.

Q. What about users under 18?

India's DPDP Act needs verifiable parental consent for minors. We need a consent flow or an 18+ rule before a public launch.

Q. Which currencies are supported?

INR only. Other currencies go to the review list.

Q. Why Argon2id and not bcrypt?

It is memory-hard, so it resists GPU cracking better, and it ships prebuilt binaries that install without a compiler.

Q. Why store money as paise?

Integers are exact. Floats cannot represent amounts like 0.1 precisely, and errors add up over many entries.

Q. Where do secrets go?

Local: .env files that are git-ignored. Production: AWS SSM Parameter Store injected into Docker Compose. Never in git.

Q. How do we run it locally?

See section 10: pnpm install, start Postgres with Docker, run migrations, then pnpm dev:api and pnpm dev:web.

14. Test cases

Done means it is verified today, by an automated test or a confirmed build. **Pending** means it still has to be written or run, by the owner shown.

Money and shared logic

ID	Case	Expected	Owner	Status
M-01	Convert "1,234.5" to paise	123450	Shared	Done
M-02	Convert "0.07"	7, no float drift	Shared	Done
M-03	Convert invalid text	null	Shared	Done
M-04	Format 123450 paise	Rupee amount with Indian grouping	Shared	Done
M-05	Amount above 2,000,000,000 paise	Rejected by schema	Backend 1	Pending

SMS parsing

ID	Case	Expected	Owner	Status
S-01	HDFC UPI debit with reference	Parsed: amount, last4, merchant, ref	Android	Done
S-02	Unknown sender	Ignored	Android	Done
S-03	OTP message	Ignored	Android	Done
S-04	Failed or declined payment	Ignored	Android	Done
S-05	Money message with unknown format	Sent to review	Android	Done
S-06	SBI debit with reference	Parsed as expense	Android	Pending
S-07	Credit alert	Parsed as income	Android	Pending
S-08	Refund or reversal	Income entry linked to original	Backend 1	Pending
S-09	Message longer than 600 chars	Ignored	Android	Pending
S-10	Promo containing the word debited	Ignored or review, never saved	Android	Pending
S-11	Non-INR amount	Sent to review	Android	Pending
S-12	Rule pattern over 500 chars from server	Rejected by schema	Android	Pending
S-13	Corpus for ICICI, Axis, Kotak, PNB, Paytm	Each sample parses correctly	Android	Pending

Duplicate detection

ID	Case	Expected	Owner	Status
D-01	Same reference number	DUPLICATE	Shared	Done
D-02	No reference, 3 seconds apart across a minute boundary	POSSIBLE, flagged	Shared	Done
D-03	Same amount, hours apart	DISTINCT	Shared	Done
D-04	Different last 4 digits	DISTINCT	Backend 1	Pending
D-05	SMS then email for the same payment	One record kept	Backend 1	Pending
D-06	Two real payments, same amount and merchant, 2 min apart	Flagged for review, not merged	Backend 1	Pending

Auth and security

ID	Case	Expected	Owner	Status
A-01	Register with short password	400	Backend 1	Done
A-02	Login with invalid email	400	Backend 1	Done
A-03	Transactions without token	401	Backend 1	Done
A-04	Forged token	401	Backend 1	Done
A-05	Valid token, invalid body	400, passes the auth guard	Backend 1	Done
A-06	Security headers present	nosniff and others set	Backend 1	Done
A-07	Register an existing email	409 email_taken	Backend 1	Pending
A-08	Register, login, call API with access token	Works end to end on a real database	Backend 1	Pending
A-09	Wrong password	401, same response as unknown email	Backend 1	Pending
A-10	Refresh rotates the token	New pair, old token revoked	Backend 1	Pending
A-11	Reuse a rotated refresh token	401 and all sessions revoked	Backend 1	Pending
A-12	Expired access token	401, client refreshes and retries	Frontend 1	Pending
A-13	More than 10 auth calls a minute	429	Backend 1	Pending
A-14	CORS from a non-listed origin	Blocked	Backend 1	Pending
A-15	Body over 100 KB	413	Backend 1	Pending
A-16	User A reads or deletes user B's transaction	Not found, nothing leaked	Backend 1	Pending
A-17	SQL injection strings in text fields	Stored as plain text, no effect	Backend 1	Pending
A-18	Delete account	All user data removed	Backend 1	Pending

Transactions and sync

ID	Case	Expected	Owner	Status
T-01	Create transaction	201 with stored record	Backend 1	Pending
T-02	Send same id again	200 with same record, no duplicate	Backend 1	Pending
T-03	Id already used by another user	409, no data leaked	Backend 1	Pending
T-04	Account or category of another user	400	Backend 1	Pending
T-05	Delete then sync	Stays deleted (soft delete)	Backend 1	Pending
T-06	Edit on two devices	Higher server version wins, old kept in history	Backend 1	Pending
T-07	Add entries offline, reconnect	All synced once, in order	Frontend 1	Pending
T-08	Retry sync after a timeout	No duplicates	Frontend 1	Pending
T-09	List with date range and limit	Correct filtering and order	Backend 1	Pending
T-10	Time zone near midnight	Stored UTC, shown in local day	Frontend 2	Pending

Budgets, goals and insights

ID	Case	Expected	Owner	Status
B-01	Spend reaches 50%, 80%, 100% of budget	One alert each, once per period	Backend 2	Pending
B-02	Alert job runs twice	No repeated alert	Backend 2	Pending
B-03	Budget cycle starts on the 5th	Spending grouped from the 5th	Backend 2	Pending
B-04	Safe to spend today	(Budget left minus upcoming recurring) divided by days left	Backend 2	Pending
B-05	Runway with zero spending	No end date, no divide by zero	Backend 2	Pending
B-06	Goal with passed deadline	No negative per-day value	Backend 2	Pending
B-07	Recurring entry job runs twice	One entry per due date	Backend 2	Pending
B-08	Month over month with empty previous month	No crash, clear message	Frontend 2	Pending

Web, PWA and UI

ID	Case	Expected	Owner	Status
W-01	Production build succeeds	dist with sw.js and manifest	Frontend 1	Done
W-02	Install on Chrome desktop and Android	Install prompt appears	Frontend 1	Pending
W-03	Add to Home Screen on iPhone	Opens standalone, push works	Frontend 1	Pending
W-04	Load app with network off	Shell and local data load	Frontend 1	Pending
W-05	Login, logout, refresh page	Session behaves correctly	Frontend 1	Pending
W-06	Phone width 360 px	No horizontal scroll	Frontend 2	Pending
W-07	Keyboard and screen reader on forms	Usable and labelled	Frontend 2	Pending
W-08	CSV export	File matches stored data	Frontend 2	Pending
W-09	Playwright: sign up, add expense, see dashboard	Passes in CI	Frontend 1	Pending

Android

ID	Case	Expected	Owner	Status
N-01	Debug APK builds	app-debug.apk produced (4.4 MB)	Android	Done
N-02	Grant SMS permission	History scan runs	Android	Pending
N-03	Deny SMS permission	App works fully without SMS	Android	Pending
N-04	Sideloaded on Android 13+	Onboarding guides restricted settings	Android	Pending
N-05	Personal chat and OTP in inbox	Never reach JavaScript	Android	Pending
N-06	New bank SMS arrives with app closed	Captured, or caught on next open	Android	Pending
N-07	Samsung, Xiaomi, Oppo battery settings	Background behaviour documented	Android	Pending
N-08	Older APK after API update	Still works on /api/v1, update prompt shown	Android	Pending

Infrastructure

ID	Case	Expected	Owner	Status
I-01	CI on pull request	Lint, typecheck, test, build, audit pass	DevOps	Pending
I-02	Docker image builds, health check passes	/health returns ok	DevOps	Pending
I-03	Deploy to production through CI only	No manual deploy path	DevOps	Pending
I-04	Restore database from backup	Data matches	DevOps	Pending
I-05	Secrets in git history	None found	DevOps	Pending
I-06	RDS reachability	Only the EC2 security group	DevOps	Pending
I-07	HTTP request	Redirects to HTTPS	DevOps	Pending

Summary

Total cases	Done	Pending
84	20	64

Priority order: auth and transaction integration tests on a real database (Backend 1), then the SMS bank corpus (Android), then budgets and alerts (Backend 2). The 16 Vitest tests cover the Done rows for shared logic, SMS, duplicates and auth guards.

15. Risks and open decisions

Risk or decision	Impact	Plan
SMS formats differ by bank and change	Missed or wrong captures	Review list, rule data served by API, corpus from the team, per-bank tests
Gmail restricted scope	Test mode only, tokens expire in 7 days	Stay within test users, prompt reconnect, apply for verification before public launch
Sideloaded APK and Android restricted settings	SMS permission looks blocked	Clear onboarding with screenshots, FAQ entry
Battery savers kill background work	Live capture is unreliable on some phones	Catch-up scan on every app open, document per brand
Single EC2 instance	Downtime if it fails	Offline mode keeps the app usable. Backups to S3. Move to a managed service if needed
Users under 18 (DPDP Act)	Legal exposure	Add consent flow or an 18+ rule before public launch
Refresh token storage on web	XSS could expose it	Strict CSP, move to secure storage on Android, review options for web
Scope is large	Late or unfinished features	Three phases, MVP first, weekly demos
OCR accuracy on poor bills	Wrong amounts	Always pre-fill and confirm, never auto-save